How many ways we can load the .class file into JVM Memory :

There are so many ways to load the .class file into JVM memory but the following are the common examples :

1) By using java command

```
public class Test
{
}

javac Test.java
java Test
```

Here we are making a request to class loader sub system to load Test.class file into JVM memory

2) By using Constructor (new keyword at the time of creating object).

3) By accessing static data member of the class.

4) By using inheritance (before sub class, super class will be loaded)

5) By using Reflection API

Loading the .class file by using Reflection API :

* There is a predefined class called **Class available in java.lang package.**

* This **Class** class has provided a static factory method called **forName(String className)** which is used to load the given .class file dynamically at runtime.

* It throws a checked exception (Wiil learn in exception handling) i.e ClassNotFoundException so provide either try catch OR declare the method as throws.

```
public static Class<T> forName(String className) throws ClassNotFoundException
```

//Program on EditPlus :

```
class Demo
{
    static
    {
        IO.println("SB");
    }
}
public class Test
{
  public static void main(String [] args) throws Exception
    {
        Class.forName("Demo");  //load Demo.class file at runtime
    }
}
```

//Program on Eclipse IDE :

```
package com.ravi.class_loading;

class Demo
{
    static
    {
        IO.println("SB");
    }
}
public class ReflectionAPI
{
    public static void main(String[] args) throws Exception
    {
        Class.forName("com.ravi.class_loading.Demo"); //FQN is required
    }
}
```

What is compile time constant :

A compile time constant is a constant that is evaluated and replaced with its value at compile time rather than runtime.

It must be declared with static and final modifier as well as must be initialized with constant expression. (Must not be initialized by method call)

At compile time, constant value will be converted by compiler at the time of compilation itself so, at runtime JVM can see the value but not the class name so class will not be loaded as shown in the program.

Example : public **static final** int A = **100**; //Valid [Here class will not be loaded by JVM]

public static final int A = m1(); //valid [Here class will be loaded by JVM]

```
//Program :
class Sample
{
  public static final int A = 100; //Compile Time Constant

  static
    {
     IO.println("Static Block");
    }
}

public class CompileTimeConstantDemo1
{
    public static void main(String[] args)
    {
        System.out.println(Sample.A);  //Sample.A will be replaced by 100
    }
}
```

```
class Foo
{
    public static final int A = m1();  //It is not a compile time constant

   public static int m1()
    {
        return 100;
    }

    static
    {
        IO.println("SB");
    }
}

public class CompileTimeConstantDemo2
{
    public static void main(String[] args)
    {
        IO.println(Foo.A);
    }
}
```

---

2 files :

Test.java

```
public class Test
{
    public static final int MAX_VALUE = 999;
}
```

Main.java

```
public class Main
{
    public static void main(String[] args)
    {
       System.out.println(Test.MAX_VALUE);
    }
}
```

Note : If we compile and execute both programs we will get 999 value, now change the 999 to 1500, Re- compile only Test.java so a new .class file will be created which will hold 1500 value. Now without re-compilation of main.java if we execute Main.java then we wil get 999 so it takes the value at compile time.

**** A static method does not act on instance variable directly why?**

All the static members (static Field, static block, static method, static nested inner class) are loaded/executed at the time of loading the .class file into JVM Memory.

At class loading phase object is not created because object is created in the 2nd phase i.e Runtime data area so at the TIME OF EXECUTION OF STATIC METHOD AT CLASS LOADING PAHSE, NON STATIC VARIABLE WILL NOT BE AVAILABLE BY DEFAULT henec we can't access non static variable from static context[static block, static method and static nested inner class] without creating the object.

```
public class Demo
{
    int x = 100;

    public static void main(String[] args)
    {
        IO.println(x);  //error
    }
}


class Test
{
    private int x;

   public Test(int x)
    {
        this.x = x;
    }

    public static void print()
    {
        IO.println("x value is :"+x);
    }

}
public class StaticDemo
{
    public static void main(String[] args)
    {
        Test t1 = new Test(100);
        Test.print();
    }
}
```

Runtime Data Areas :

* It is also known as memory areas.

* Once a class is loaded in the JVM then based on the content We have total 5 types of memories :

1) Method Area [Only one method area per JVM]

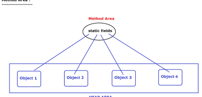
2) HEAP Area [Only one method area per JVM]

3) Stack Area [We can have multiple statck area for a single JVM]

4) PC Register

5) Native Method Stack.

Method Area :



Method Area :

Whenever a class is loaded then the class is dumpped inside method area and returns java.lang.Class class object.

It provides all the information regarding the class, like name of the class, name of the package, static and non static fields available in the class, methods available in the class and so on.

We have only one method area per JVM that means for a single JVM we have only one Method area.

This Method Area OR Class Area is sharable by all the objects.

HEAP AREA :

Whenever we create an object in java then the properties and behavior of the object are strored in a special memory area called HEAP AREA.

We have only one HEAP AREA per JVM.

STACK Area :

All the methods are executed as a part of Stack Area.

Whenever we call a method in java then internally one stack Frame will be created to hold method related information.

Every Stack frame contains 3 parts :
1) Local Variable arrays
2) Frame Data
3) Operand Stack.

We have multiple stack area for a single JVM.

Everytime we create a new thread in java then JVM will create a separate Runtime Stack Memory.[Multithreading]